

Federated Learning for Credit Card Fraud Detection Across Simulated Banks

Indra karan Nukala

26-07-2026

Contents

1	Introduction	3
1.1	Research Questions	3
1.2	Project Contributions	3
2	Related Work	4
2.1	Federated Learning in Financial Fraud Detection	4
2.2	Handling Class Imbalance	4
2.3	Alternative Federated Aggregation Strategies	4
2.4	Privacy Guarantees and Their Limits	5
2.5	Comparable Machine Learning Pipelines	5
2.6	Positioning of This Project	5
3	Methodology	5
3.1	Data Understanding	5
3.1.1	Exploratory Data Analysis	6
3.2	Data Preprocessing	6
3.3	Simulated Bank Partitioning (Non-IID Client Split)	6
3.4	Local Model Architecture	6
3.5	Federated Training with Flower (FedAvg)	7
3.6	Centralized Baseline	7
3.7	Evaluation Metrics	7
4	Experiments	8
4.1	Experimental Setup	8
4.2	Experimental Procedure	8
4.3	Centralized Baseline Procedure	8
4.4	Reproducibility	8
5	Results	9
5.1	Federated Model Performance: Round 5 vs. Round 10	9
5.2	Federated vs. Centralized Baseline	9
5.3	Interpretation of the Precision-Recall Trade-off	9
5.4	Summary of Key Findings	10

6	Discussion	10
6.1	Revisiting the Research Questions	10
6.2	Effects of Non-IID Data and Class Imbalance	10
6.3	Benchmark Comparison	11
6.4	Limitations	11
7	Conclusion and Future Work	12
7.1	Summary of Findings	12
7.2	Practical Implications	12
7.3	Future Work	12
7.4	Closing Remarks	12
8	Implementation Details	13
8.1	Technology Stack	13
8.2	Project Structure	13
8.3	Running the Pipeline	13
9	Ethical and Regulatory Considerations	13
9.1	Data Privacy Motivation	13
9.2	Residual Privacy Risks in Federated Learning	13
9.3	Fairness Considerations	14
9.4	Deployment Considerations	14
10	References	14

1 Introduction

Credit card fraud is a persistent and costly problem for financial institutions worldwide, with global card fraud losses estimated in the tens of billions of dollars annually. Traditional fraud detection systems rely on centralized machine learning, where transaction data from multiple banks or branches is pooled into a single location before a model is trained. While centralized training is straightforward and often yields strong predictive performance, it is increasingly difficult to justify or even legally permit in the current regulatory landscape. Regulations such as the GDPR in the European Union and various national data-localization laws restrict the transfer and pooling of sensitive financial data across institutional or geographic boundaries. Beyond regulation, banks are also reluctant to share proprietary transaction data with competitors, even indirectly through a shared model-training pipeline, due to competitive and reputational concerns.

Federated Learning (FL) offers a solution to this dilemma. Instead of moving data to a central server, FL moves the model to the data: each participating institution trains a local copy of a shared model on its own data, and only the resulting model updates (not the raw data) are sent to a central server for aggregation. This project applies Federated Learning to the problem of credit card fraud detection by simulating four independent banks, each holding a distinct, non-overlapping partition of a real-world credit card transaction dataset. Using the Flower federated learning framework in combination with TensorFlow/Keras, a shared fraud-detection model is trained collaboratively across the four simulated banks over multiple communication rounds, without any bank ever exposing its raw transaction records to the others or to the central server.

The goal of this project is to demonstrate that a Federated Learning pipeline, built entirely from open-source tools, can achieve strong fraud-detection performance while preserving the privacy guarantees that make FL attractive for the financial sector. Specifically, the project evaluates how detection performance evolves across federated training rounds, how the model performs relative to a centralized baseline trained on the same combined data, and what practical trade-offs (particularly between Precision and Recall) emerge from the federated approach under significant class imbalance.

1.1 Research Questions

- **RQ1:** How does a federated fraud-detection model trained across four simulated banks compare to a centralized model trained on the same pooled data?
- **RQ2:** How does the federated global model’s performance evolve across successive communication rounds (Round 1 through Round 10)?
- **RQ3:** What Precision-Recall trade-off emerges in the federated model, and what does this imply for real-world deployment in a bank’s fraud operations?

1.2 Project Contributions

- An end-to-end, reproducible Federated Learning pipeline for fraud detection, using Flower and TensorFlow, built on a well-known public dataset.
- A stratified, non-IID data partitioning scheme that simulates four realistic bank clients while preserving each client’s local fraud ratio.
- A systematic round-by-round evaluation of the federated global model using Accuracy, Precision, Recall, F1-score, and ROC-AUC.

- A discussion of the Precision-Recall trade-off and its practical implications for real-world bank deployment.

2 Related Work

Federated Learning was first formally introduced by McMahan et al. (2017) at Google, who proposed the Federated Averaging (FedAvg) algorithm to train machine learning models across many mobile devices without centralizing user data. This work established FL as a viable alternative to centralized training whenever data cannot or should not be pooled, and FedAvg remains the most widely used federated aggregation strategy today. It is the same strategy adopted for the global model aggregation in this project’s Flower-based pipeline.

Since McMahan et al.’s original work, FL research has diverged into two broad application settings: cross-device FL, involving potentially millions of resource-constrained clients such as smartphones, and cross-silo FL, involving a small number of powerful, reliable participants such as hospitals, banks, or research institutions. This project falls into the cross-silo category, since each simulated bank is treated as a stable, always-available participant holding a substantial local partition of data, consistent with real-world financial institutions.

2.1 Federated Learning in Financial Fraud Detection

Financial fraud detection is frequently cited in the literature as an ideal use case for FL, precisely because strict regulatory barriers – such as GDPR in Europe and various data localization laws – legally prevent banks from pooling raw transaction records for joint model training. Several prior studies have explored FL for fraud and anomaly detection using FedAvg and its variants, generally reporting that federated models can approach, though do not always fully match, the performance of centralized models trained on pooled data. A consistent finding across this body of work is that severe class imbalance – rather than data heterogeneity across clients – is often the dominant challenge affecting the quality of fraud-detection models, federated or otherwise.

2.2 Handling Class Imbalance

Class imbalance is a well-studied problem in centralized machine learning. Common mitigation strategies include oversampling techniques such as SMOTE, undersampling of the majority class, and cost-sensitive learning via class weighting. In a federated context, applying these techniques is complicated by the fact that each client’s local class distribution may differ meaningfully from the global distribution – a phenomenon known as label distribution skew. This project addresses the imbalance problem primarily through per-client class weighting during local training, combined with a stratified partitioning scheme that keeps each simulated bank’s fraud ratio broadly consistent with the overall dataset.

2.3 Alternative Federated Aggregation Strategies

While FedAvg is the baseline strategy used throughout this project, several alternative aggregation algorithms have been proposed in the literature to address its known limitations under non-IID data. FedProx (Li et al., 2020) introduces a proximal term to the local objective function that limits how far local models can drift from the global model, improving convergence stability. FedNova normalizes client updates to account for differing numbers of local training steps across clients. FedAvgM adds server-side momentum to smooth the aggregation process across communication rounds. None of these alternatives were implemented in this project, but they are identified in Section 7 as promising directions for future work.

2.4 Privacy Guarantees and Their Limits

It is important to note that Federated Learning, on its own, does not provide formal, mathematically guaranteed privacy. Raw data never leaves each participant’s local environment, which prevents the most direct form of data leakage, but the model updates exchanged during training (gradients or weights) can, under certain conditions, be exploited through techniques such as gradient inversion attacks to partially reconstruct training examples. Stronger, provable privacy guarantees typically require combining FL with additional techniques such as Differential Privacy (adding calibrated statistical noise to updates) or Secure Aggregation (cryptographic protocols that combine client updates such that the server never observes any individual client’s raw contribution). Neither technique was implemented in this project, representing an important direction for future work discussed in Section 7.

2.5 Comparable Machine Learning Pipelines

Related work on data-driven detection pipelines outside the fraud domain reinforces several of the design choices made in this project. For example, research on leak detection in water distribution networks has shown that ensemble tree-based models such as Random Forest and XGBoost consistently outperform simpler linear or distance-based models such as Logistic Regression and K-Nearest Neighbors when working with structured, tabular sensor data (Meleshko et al., 2020; Park et al., 2024). Although this project uses a neural network (a fully-connected feed-forward architecture) rather than a tree-based ensemble, similar considerations around feature informativeness, class balance, and systematic multi-metric evaluation (accuracy, F1-score, and ROC-AUC) informed the experimental design described in Section 3.

2.6 Positioning of This Project

This project contributes an applied, end-to-end demonstration of FedAvg-based fraud detection using an open, well-known public dataset, combined with a systematic round-by-round evaluation and comparison against a centralized baseline. Unlike much of the academic literature, which often reports results on custom or proprietary financial datasets that cannot be independently verified, this project uses the publicly available Kaggle Credit Card Fraud dataset, making the full pipeline reproducible by other researchers or students.

3 Methodology

This project builds a complete Federated Learning pipeline for credit card fraud detection using the Flower (flwr) framework for orchestration and TensorFlow/Keras for local model training. The overall workflow consists of five stages: data understanding and preprocessing, non-IID client partitioning, local model design, federated training via the FedAvg strategy, and round-by-round evaluation against a centralized baseline.

3.1 Data Understanding

The dataset used is the well-known Kaggle Credit Card Fraud Detection dataset, containing anonymized European credit card transactions made over two days in September 2013. The dataset contains 284,807 transactions in total, of which only 492 are labeled as fraudulent – a fraud rate of approximately 0.17%, making this an extreme class-imbalance problem. Each transaction is described by 30 input features: 28 anonymized principal components (V1 through V28) generated via PCA to protect cardholder identity, along with the transaction Amount and Time elapsed since the first transaction in the dataset. The target variable, Class, is binary (0 = legitimate, 1 = fraud).

3.1.1 Exploratory Data Analysis

Exploratory analysis confirmed the extreme imbalance between the two classes: legitimate transactions outnumber fraudulent ones by roughly 577 to 1. Distribution plots of the anonymized PCA features (V1–V28) showed that several components, notably V14, V12, V10, and V17, exhibit visibly different distributions between fraud and non-fraud transactions, suggesting they carry strong discriminative signal for the downstream classifier. The Amount feature was found to be highly right-skewed, with the vast majority of transactions under 100 monetary units and a long tail of larger transactions, motivating the use of feature scaling prior to training.

3.2 Data Preprocessing

- **Feature scaling:** The Amount and Time columns were standardized using z-score normalization (StandardScaler), since the PCA-derived V1–V28 features are already approximately standardized.
- **Train/test split:** The dataset was split into training and test sets using an 80/20 stratified split, preserving the original fraud ratio in both partitions.
- No synthetic oversampling (e.g., SMOTE) was applied at the global level; class imbalance was instead handled locally at each client via class weighting during training, described in Section 3.4.

3.3 Simulated Bank Partitioning (Non-IID Client Split)

To realistically simulate four independent banks participating in a federated fraud-detection system, the training data was partitioned into four disjoint, non-overlapping client shards. Rather than a purely random (IID) split, a stratified partitioning strategy was used: each client shard was constructed to preserve approximately the same fraud ratio as the global training set, while introducing controlled variation in shard size and feature distribution across clients to emulate the natural heterogeneity of real bank transaction volumes. This produces a mildly non-IID setting – realistic for cross-silo financial FL, where institutions differ in size and customer base, but not so extreme as to prevent convergence of the FedAvg aggregation strategy.

Client	Role	Approx. Records	Fraud Ratio
Bank 1	FL Client 1	~56,900	~0.17%
Bank 2	FL Client 2	~56,900	~0.17%
Bank 3	FL Client 3	~56,900	~0.17%
Bank 4	FL Client 4	~56,900	~0.17%

Table 1: Simulated bank client partitions.

3.4 Local Model Architecture

Each simulated bank trains an identical local model architecture, a fully-connected feed-forward neural network implemented in TensorFlow/Keras. The architecture consists of an input layer accepting the 30 transaction features, followed by two hidden dense layers (64 and 32 units respectively) with ReLU activations and dropout regularization (rate 0.3) to reduce overfitting, and a final single-unit output layer with a sigmoid activation for binary classification. The model is compiled with the Adam optimizer and binary cross-entropy loss. To counteract the severe class imbalance during local training, per-class weights (`class_weight`) are computed for each client’s local shard and passed to the Keras `fit()` call, penalizing misclassification of the minority (fraud) class more heavily than the majority class.

Layer	Units	Activation	Notes
Input	30	–	Standardized features
Dense 1	64	ReLU	–
Dropout 1	–	–	rate = 0.3
Dense 2	32	ReLU	–
Dropout 2	–	–	rate = 0.3
Output	1	Sigmoid	Binary fraud probability

Table 2: Local model architecture.

3.5 Federated Training with Flower (FedAvg)

The Flower framework orchestrates the federated training process. A Flower server process manages the global model and coordinates 10 communication rounds. In each round, the server (1) sends the current global model weights to all four connected clients, (2) each client trains locally on its own data shard for a fixed number of local epochs (5 epochs per round, batch size 32), (3) each client sends its updated model weights back to the server, and (4) the server aggregates the four sets of client weights using the Federated Averaging (FedAvg) algorithm, which computes a weighted average of client parameters proportional to the number of training examples held by each client. The resulting aggregated weights become the new global model, which is then evaluated and broadcast to clients at the start of the next round.

This project uses Flower’s simulation mode, in which all four clients and the server run within a single process/machine, communicating via Flower’s internal messaging rather than over a real network. This allows the entire federated pipeline to be developed, tested, and evaluated without requiring physically separate infrastructure for each simulated bank, while still faithfully reproducing the FedAvg communication pattern that would occur in a real multi-institution deployment.

3.6 Centralized Baseline

To answer RQ1, a centralized baseline model was also trained using the identical network architecture described in Section 3.4, but trained on the full pooled training set (all four bank shards combined) in a single, non-federated training run. This baseline uses the same number of total training epochs as the federated run (5 local epochs $\times$ 10 rounds = 50 effective epochs) to ensure a fair comparison of total training exposure between the centralized and federated approaches.

3.7 Evaluation Metrics

Given the extreme class imbalance in the dataset, accuracy alone is a poor and misleading indicator of model quality – a naive model that always predicts ”legitimate” would already achieve roughly 99.83% accuracy while catching zero fraud cases. For this reason, the model is evaluated using five complementary metrics after every communication round:

- **Accuracy:** The proportion of all transactions (fraud and legitimate) correctly classified.
- **Precision:** Of all transactions flagged as fraud, the proportion that are actually fraudulent. Low precision means many false alarms.
- **Recall (Sensitivity):** Of all actual fraud transactions, the proportion correctly identified. Low recall means fraud cases are being missed.
- **F1-score:** The harmonic mean of Precision and Recall, providing a single balanced measure especially useful under class imbalance.

- **ROC-AUC:** The area under the Receiver Operating Characteristic curve, measuring the model’s ability to discriminate between fraud and legitimate transactions across all classification thresholds.

4 Experiments

This section describes the practical execution of the federated training pipeline, the software and hardware environment, and the specific configuration choices made during experimentation.

4.1 Experimental Setup

Component	Configuration
FL Framework	Flower (flwr), simulation mode
Model Framework	TensorFlow / Keras
Number of Clients	4 (simulated banks)
Aggregation Strategy	FedAvg
Communication Rounds	10
Local Epochs per Round	5
Batch Size	32
Optimizer	Adam
Loss Function	Binary Cross-Entropy
Train/Test Split	80% / 20% (stratified)
Class Imbalance Handling	Per-client class_weight

Table 3: Experimental setup configuration.

4.2 Experimental Procedure

For each of the 10 communication rounds, the following procedure was executed: the server distributed the current global model weights to all four clients; each client trained locally for 5 epochs on its own shard using class-weighted binary cross-entropy loss; each client returned its updated weights along with its local training sample count; the server aggregated all four updates via FedAvg, weighting each client’s contribution proportionally to its number of training examples; and the newly aggregated global model was evaluated on the held-out global test set, with Accuracy, Precision, Recall, F1-score, and ROC-AUC recorded for that round. This procedure was repeated identically for all 10 rounds, and the resulting metrics history forms the basis for the round-by-round analysis in Section 5.

4.3 Centralized Baseline Procedure

The centralized baseline model was trained separately, using the same architecture, optimizer, loss function, and total epoch budget (50 epochs) as the federated pipeline, but trained in a single uninterrupted run on the full pooled training set rather than across four separate client shards. This provides a like-for-like comparison point for RQ1, isolating the effect of federated (distributed, privacy-preserving) training from a traditional centralized approach while holding model capacity and total training exposure constant.

4.4 Reproducibility

All experiments were implemented in Python using the Flower and TensorFlow libraries, with fixed random seeds used for data splitting and model weight initialization to support repro-

ducibility. The complete pipeline, including data preprocessing scripts, client and server application code, and the simulation runner, is organized as a self-contained project (see `run_simulation.py` and associated modules) that can be executed end-to-end with a single command after installing the dependencies listed in `requirements.txt`.

5 Results

This section presents the round-by-round performance of the federated global model over 10 communication rounds, alongside a comparison against the centralized baseline model described in Section 3.

5.1 Federated Model Performance: Round 5 vs. Round 10

Metrics were confirmed at two key checkpoints during federated training: an early-to-mid checkpoint at Round 5, and the final checkpoint at Round 10. The comparison below highlights how the model’s behavior shifted over the second half of training.

Metric	Round 5	Round 10	Change
Accuracy	99.62%	99.69%	+0.07 pp
Precision	18.02%	34.25%	+16.23 pp
Recall	89.80%	88.78%	−1.02 pp
F1-score	0.3002	0.4943	+0.1941
ROC-AUC	0.9701	0.9793	+0.0092

Table 4: Federated global model performance at Round 5 vs. Round 10.

The results show that Recall remained essentially stable between Round 5 (89.80%) and Round 10 (88.78%), a marginal decrease of just over one percentage point. In contrast, Precision nearly doubled, rising from 18.02% to 34.25%. This pattern indicates that the additional training rounds primarily helped the global model reduce false positives – transactions incorrectly flagged as fraud – rather than uncover additional true fraud cases that were previously missed. The F1-score, which balances Precision and Recall, improved substantially from 0.3002 to 0.4943, reflecting this improved balance. ROC-AUC also improved modestly from 0.9701 to 0.9793, indicating a slight overall improvement in the model’s ability to discriminate between fraud and legitimate transactions across all thresholds.

5.2 Federated vs. Centralized Baseline

A direct, fully-confirmed comparison against the centralized baseline model is one of the areas flagged for completion in this project (see Section 6.4, Limitations). Based on the federated model’s strong standalone ROC-AUC of 0.9793 at Round 10, and consistent with patterns reported in the broader federated learning literature (Section 2.1), the federated approach is expected to approach, though not necessarily exceed, the performance of a centralized model trained on the same pooled data. The centralized baseline run (Section 3) was configured with an identical architecture and total training budget specifically to enable this comparison once fully confirmed.

5.3 Interpretation of the Precision-Recall Trade-off

The Round 10 results show a model that strongly favors Recall (88.78%) over Precision (34.25%). In practical terms, this means the model successfully identifies approximately 9 out of every 10 fraudulent transactions in the test set, but only about 1 in 3 transactions flagged as fraud is

Metric	Federated (Round 10)	Centralized Baseline
Accuracy	99.69%	[to be confirmed]
Precision	34.25%	[to be confirmed]
Recall	88.78%	[to be confirmed]
F1-score	0.4943	[to be confirmed]
ROC-AUC	0.9793	[to be confirmed]

Table 5: Federated (confirmed) vs. Centralized baseline (pending confirmation) results.

actually fraudulent. For a real-world fraud detection system, this trade-off is generally desirable as a starting point: missing fraud (a false negative) is typically far more costly to a bank and its customers than investigating a false alarm (a false positive). This favors a high-Recall design, with the expectation that flagged transactions are routed to human analysts or secondary automated checks rather than being auto-blocked outright given the current Precision level.

5.4 Summary of Key Findings

- The federated model reached 99.69% accuracy, 34.25% Precision, 88.78% Recall, F1-score 0.4943, and ROC-AUC 0.9793 by Round 10.
- Recall plateaued early (Round 5 to Round 10 change of only -1.02 percentage points), suggesting most fraud-catching capability was learned within the first half of training.
- Precision nearly doubled across the same period, indicating later rounds primarily reduced false positives.
- ROC-AUC of 0.9793 indicates strong overall discriminative ability between fraud and legitimate transactions.
- A full centralized-vs-federated comparison and complete 10-round metrics history remain to be finalized (see Section 6.4).

6 Discussion

6.1 Revisiting the Research Questions

- **RQ1 (Federated vs. Centralized):** Partially answered. The federated model demonstrates strong standalone performance (ROC-AUC 0.9793), but a fully confirmed centralized baseline comparison is still pending, as noted in Section 6.4.
- **RQ2 (Evolution across rounds):** Answered for the confirmed checkpoints (Round 5 and Round 10). Recall plateaued early in training, while Precision improved substantially over the second half of training.
- **RQ3 (Precision-Recall trade-off):** Answered. The federated model favors Recall over Precision, consistent with prioritizing fraud-detection coverage over minimizing false alarms – an appropriate default for a fraud-screening system.

6.2 Effects of Non-IID Data and Class Imbalance

Although the stratified partitioning scheme in Section 3 was designed to preserve each simulated bank’s local fraud ratio close to the global average, some residual heterogeneity across clients is expected simply due to differences in shard size and the natural variability of PCA-derived

features across random subsets of transactions. This mild non-IID setting is realistic for cross-silo financial FL and did not prevent the FedAvg strategy from converging to a strong global model, but it likely contributed to the relatively low Precision observed in earlier rounds, since a client-specific decision boundary learned on one bank’s data may not transfer perfectly to another bank’s transaction patterns until several rounds of averaging have occurred.

The extreme class imbalance in the underlying dataset (0.17% fraud rate) remains the single most significant modeling challenge in this project, more so than the federated setting itself. Per-client class weighting was effective at pushing Recall to a strong level early in training, but the corresponding cost – a large number of false positives – required additional rounds of training to partially resolve, as reflected in the Precision improvement between Round 5 and Round 10.

6.3 Benchmark Comparison

While a fully confirmed centralized baseline is not yet available (Section 6.4), the federated model’s ROC-AUC of 0.9793 compares favorably with commonly cited benchmarks for the same Kaggle Credit Card Fraud dataset in the broader literature, where centralized models (including tree-based ensembles and neural networks) typically report ROC-AUC values in the 0.95–0.99 range depending on preprocessing and resampling strategy. This suggests the federated approach used in this project is competitive with typical centralized results reported for this dataset, even without a direct internal baseline confirmed.

6.4 Limitations

- **Incomplete metrics history:** Only Round 5 and Round 10 metrics were fully confirmed; a complete round-by-round trend (Rounds 1–10) would provide a clearer picture of convergence dynamics.
- **Centralized baseline not yet confirmed:** The centralized model was trained under an identical configuration, but its final metrics were not confirmed at the time of writing, preventing a complete answer to RQ1.
- **Simulated, not physically distributed, clients:** All four banks were simulated within Flower’s single-process simulation mode rather than deployed across physically separate machines or networks, so real-world network latency, client dropout, and communication overhead were not evaluated.
- **Only one aggregation strategy tested:** FedAvg was the only federated strategy evaluated; alternatives such as FedProx may behave differently under the non-IID conditions present here.
- **Fixed hyperparameters:** Local epochs (5), batch size (32), and rounds (10) were fixed based on reasonable defaults rather than being tuned via systematic hyperparameter search.
- **No formal privacy guarantee:** As discussed in Section 2.4, FedAvg alone does not provide provable privacy guarantees against gradient-based reconstruction attacks; Differential Privacy or Secure Aggregation would be required for stronger guarantees.

7 Conclusion and Future Work

7.1 Summary of Findings

This project built a complete Federated Learning pipeline, using Flower and TensorFlow, to detect credit card fraud across four simulated banks without sharing raw transaction data. By Round 10, the federated global model achieved 99.69% accuracy, 34.25% Precision, 88.78% Recall, an F1-score of 0.4943, and a ROC-AUC of 0.9793. Between Round 5 and Round 10, Recall remained stable while Precision nearly doubled, indicating that later training rounds mainly helped the model reduce false positives rather than uncover additional fraud patterns. These results suggest that Federated Learning, when combined with stratified data partitioning and class weighting, can produce a fraud detection model with strong fraud-catching ability (high Recall) while preserving the privacy of each bank’s transaction data.

7.2 Practical Implications

These findings suggest that banks could realistically deploy a federated fraud detection system that catches roughly 9 out of every 10 fraud cases, without ever pooling sensitive customer data. In practice, such a system would flag transactions for human review rather than auto-blocking them, given the current Precision level, meaning a portion of flagged transactions would be false alarms requiring manual verification. As Precision continues to improve with additional rounds or tuning, the system could eventually support more automated decisioning.

7.3 Future Work

- Complete the full 10-round metrics history and produce a round-by-round trend chart to better understand training dynamics.
- Confirm and report centralized baseline results for a rigorous federated-vs-centralized comparison.
- Experiment with alternative federated strategies (e.g., FedProx, FedAvgM) to handle non-IID data more robustly.
- Test with a larger and more diverse number of simulated banks to study scalability.
- Explore adaptive class-weighting or resampling strategies to further improve Precision without sacrificing Recall.
- Validate the approach on additional, real-world-style fraud datasets to test generalization.
- Incorporate Differential Privacy or Secure Aggregation to strengthen the formal privacy guarantees of the system.

7.4 Closing Remarks

This project demonstrates that privacy-preserving collaboration between financial institutions is not only theoretically possible but practically achievable using existing open-source tools. As data privacy regulations continue to tighten globally, Federated Learning approaches like the one presented in this report offer a promising path forward for cross-institutional fraud detection.

8 Implementation Details

8.1 Technology Stack

Component	Technology
FL Orchestration	Flower (flwr)
Model Training	TensorFlow / Keras
Data Processing	pandas, NumPy, scikit-learn
Visualization	Matplotlib / Plotly
Language	Python 3.10+
Dataset	Kaggle Credit Card Fraud (creditcard.csv)

Table 6: Technology stack used in the project.

8.2 Project Structure

The codebase is organized as a self-contained, reproducible project. Key components include a data preprocessing module that loads and cleans `creditcard.csv` and performs the stratified train/test split and feature scaling described in Section 3; a client partitioning module implementing the non-IID stratified shard split across four simulated banks; Flower client and server application modules defining the local training loop and the FedAvg-based server strategy respectively; and a single orchestration script, `run_simulation.py`, that wires together the server, all four clients, and the evaluation/metrics logging pipeline into one end-to-end runnable simulation.

8.3 Running the Pipeline

After installing the dependencies listed in `requirements.txt` and placing the `creditcard.csv` dataset file in the expected data directory, the entire federated simulation – including data preprocessing, client partitioning, 10 rounds of FedAvg training across 4 clients, and metrics evaluation – is launched with a single command: `python run_simulation.py`. Metrics for each communication round are logged and can be exported for further analysis or plotting, as summarized in Section 5.

9 Ethical and Regulatory Considerations

9.1 Data Privacy Motivation

The core motivation for adopting a federated approach in this project is rooted in real regulatory and ethical constraints faced by financial institutions. Regulations such as the EU General Data Protection Regulation (GDPR) impose strict limits on the cross-border transfer and pooling of personal financial data, and many jurisdictions additionally require certain categories of financial data to remain within national borders (data localization). Even in the absence of legal barriers, banks are commercially reluctant to share transaction-level data with competitors, since such data could reveal proprietary insights about customer behavior, merchant relationships, or business volume.

9.2 Residual Privacy Risks in Federated Learning

While Federated Learning avoids the most direct privacy risk (centralizing raw data), it is not risk-free. Research has shown that model updates exchanged during FL – particularly gradi-

ents from models trained on small batches – can, under some conditions, be used to partially reconstruct or infer properties of the underlying training data through gradient inversion or membership inference attacks. For a production deployment of the system described in this report, additional privacy-enhancing techniques would be strongly recommended: Differential Privacy, which adds calibrated noise to model updates to provide formal privacy guarantees at a small cost to model accuracy, and Secure Aggregation, a cryptographic protocol ensuring the central server only ever observes the sum or average of all client updates, never any individual client’s raw update.

9.3 Fairness Considerations

Because each simulated bank in this project draws from the same underlying population distribution (different random shards of the same overall dataset), fairness concerns related to systematic differences in customer demographics across institutions were not directly evaluated. In a real multi-bank deployment, banks may serve meaningfully different customer populations, and a shared global model could inadvertently perform worse for certain banks or customer segments if their data distribution differs significantly from the federated average. Future work should evaluate per-client (per-bank) performance in addition to global metrics, to ensure the shared model does not systematically underperform for any single participating institution.

9.4 Deployment Considerations

Given the current Precision level (34.25%) at Round 10, a production deployment of this model should not be used to automatically block transactions. Instead, flagged transactions should be routed to a human fraud-review queue or a secondary verification step (e.g., an SMS confirmation to the cardholder), consistent with common industry practice for models still in the early stages of Precision refinement.

10 References

1. McMahan, B., Moore, E., Ramage, D., Hampson, S. and y Arcas, B.A., 2017. Communication-efficient learning of deep networks from decentralized data. *Proceedings of AISTATS 2017*.
2. Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A. and Smith, V., 2020. Federated optimization in heterogeneous networks. *Proceedings of MLSys 2020*.
3. Beutel, D.J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K.H., Parcollet, T., de Gusmao, P.P.B. and Lane, N.D., 2020. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*.
4. Dal Pozzolo, A., Caelen, O., Johnson, R.A. and Bontempi, G., 2015. Calibrating probability with undersampling for unbalanced classification. *IEEE Symposium Series on Computational Intelligence*.
5. Vrachimis, S.G., Kyriakou, M.S., Eliades, D.G. and Polycarpou, M.M., 2018. LeakDB: A benchmark dataset for leakage diagnosis in water distribution networks. *Proc. of the WDSA/CCWI Joint Conference (Vol. 1)*.
6. Meleshko, A.V., Desnitsky, V.A. and Kotenko, I.V., 2020. Machine learning based approach to detection of anomalous data from sensors in cyber-physical water supply systems. *IOP Conference Series: Materials Science and Engineering*, 709(3), p.033034.

7. Park, H., Oh, Y., Kim, K., Kim, T., Hyung, J. and Koo, J., 2024. Optimal leakage detection and classification of the water distribution network based on the machine learning approach. *Water Supply*, 24(6), pp.2158-2171.
8. Kairouz, P., McMahan, H.B., Avent, B., et al., 2021. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1-2), pp.1-210.
9. Bonawitz, K., Ivanov, V., Kreuter, B., et al., 2017. Practical secure aggregation for privacy-preserving machine learning. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*.